

Chapter 64: Virtual Scroll

Scenario 1: Rendering 100,000 rows without freezing the browser

Situation: A financial-reporting dashboard needs to display a table of 100,000 transaction rows. The current implementation uses `@for` over the full array inside a plain scrollable `div`, and the tab hangs for several seconds on load, then scrolling is janky at 5-10 fps.

Question: How would you fix this using Angular CDK, and what exactly makes it fast?

Answer: The root problem is that Angular is creating 100,000 real DOM nodes (and, if each row has a few cells with bindings, several hundred thousand bindings) up front. The browser has to layout, paint, and keep all of that in the render tree even though only ~20 rows are ever visible. `@angular/cdk/scrolling` solves this by only ever instantiating the DOM nodes for the rows currently in (or just outside) the viewport, and recycling them as the user scrolls — this is the same trick used by native mobile list views (`UITableView` / `RecyclerView`).

Import `ScrollingModule` (standalone: import `CdkVirtualScrollViewport` and `CdkVirtualForOf` / `CdkFixedSizeVirtualScroll` directives directly) and give the viewport an explicit height and `itemSize`:

```
import { Component, signal } from '@angular/core';
import { ScrollingModule } from '@angular/cdk/scrolling';

interface Txn { id: number; amount: number; date: string; }

@Component({
  selector: 'app-txn-table',
  standalone: true,
  imports: [ScrollingModule],
  template: `
    <cdk-virtual-scroll-viewport itemSize="48" class="viewport">
      <div *cdkVirtualFor="let txn of transactions(); trackBy: trackById" class="row">
        <span>{{ txn.id }}</span>
        <span>{{ txn.amount | currency }}</span>
        <span>{{ txn.date }}</span>
      </div>
    </cdk-virtual-scroll-viewport>
  `,
  styles: [
    `.viewport { height: 600px; width: 100%; }
    .row { height: 48px; display: flex; align-items: center; gap: 1rem; }`
  ],
})
export class TxnTableComponent {
  transactions = signal<Txn[]>(generateRows(100_000));
  trackById = (_, number, t: Txn) => t.id;
}
```

Key points to raise in the interview:

- `itemSize` must match the actual rendered row height (via CSS) or you get visual gaps/overlaps — this is the `FixedSizeVirtualScrollStrategy`.
- `trackBy` is still essential even with virtual scroll — without it, Angular tears down and rebuilds the DOM node instead of reusing/patching it during recycling, defeating half the performance benefit.
- The viewport needs a bounded height (or `autosize` /flex constraints) — CDK virtual scroll can't compute what's "visible" in an unbounded container.
- Only render the rows CDK gives you; don't also apply `*ngIf` /pagination logic on top that re-filters 100k items on every scroll tick — memoize/derive filtered arrays with a `computed()` signal instead of recomputing in the template.

Interviewer intent: Confirms the candidate understands *why* virtual scrolling works (DOM node count, not "React-style virtual DOM"), not just that "you add cdk-virtual-scroll-viewport."

Scenario 2: Variable-height rows breaking the fixed-size strategy

Situation: A social-media style feed has posts of wildly different heights (text-only posts are 80px, posts with images are 400px, posts with embedded videos vary further). Using `itemSize` with an average estimate causes visible jumping and overlapping content as the user scrolls.

Question: How do you handle variable-height items in CDK virtual scroll, and what are the tradeoffs versus a fixed-size approach?

Answer: `CdkFixedSizeVirtualScroll` assumes every item is exactly the same height — it's a simple, extremely fast strategy because computing scroll offsets is $O(1)$ arithmetic (`index * itemSize`). Variable-height content requires a different `VirtualScrollStrategy` implementation, since CDK's `ViewportRuler` / `scrollStrategy` needs to know (or estimate) an offset for arbitrary indices.

Two practical options:

1. **autosize strategy** (`@angular/cdk/scrolling` ships `CdkAutoSizeVirtualScroll` via the `autosize` directive input) — it measures rendered items after they're placed and adjusts its internal size table. This is the pragmatic default for moderately variable content:

```
@Component({
  standalone: true,
  imports: [ScrollingModule],
  template: `
    <cdk-virtual-scroll-viewport autosize class="viewport" minBufferPx="800"
    maxBufferPx="1200">
      <div *cdkVirtualFor="let post of posts(); trackBy: trackById" class="post-card">
        <app-post-card [post]="post" />
      </div>
    </cdk-virtual-scroll-viewport>
  `,
})
export class FeedComponent {
  posts = signal<Post[]>([]);
}
```

```
trackById = (_, number, p: Post) => p.id;
}
```

`autosize` is more expensive per-scroll-tick than fixed size because it has to measure DOM (`getBoundingClientRect`) to refine estimates, and it can still show a brief layout shift the first time an unmeasured item scrolls into view (classic "size estimate was wrong, then corrected" jump). It also does not support scrolling to an arbitrary far-off index reliably, since it doesn't know real heights of unrendered items.

2. **Custom `VirtualScrollStrategy`** — when you actually know the height category up front (e.g., post has `hasImage: boolean`, `hasVideo: boolean`), you can precompute exact or near-exact heights and implement your own strategy extending the `VirtualScrollStrategy` interface, tracking a prefix-sum array of offsets:

```
import { VirtualScrollStrategy } from '@angular/cdk/scrolling';
import { Observable, Subject } from 'rxjs';

export class KnownSizeVirtualScrollStrategy implements VirtualScrollStrategy {
  private readonly _scrolledIndexChange = new Subject<number>();
  scrolledIndexChange: Observable<number> = this._scrolledIndexChange;
  private offsets: number[] = []; // prefix sums
  private viewport: any;

  constructor(private itemHeights: number[]) {
    this.computeOffsets();
  }

  private computeOffsets() {
    let sum = 0;
    this.offsets = this.itemHeights.map(h => (sum += h, sum));
  }

  attach(viewport: any): void {
    this.viewport = viewport;
    this.updateContent();
  }

  detach(): void { this._scrolledIndexChange.complete(); }
  onDataLengthChanged(): void { this.computeOffsets(); this.updateContent(); }
  onContentScrolled(): void { this.updateContent(); }
  onContentRendered(): void {}
  onRenderedOffsetChanged(): void {}
  scrollToIndex(index: number, behavior: ScrollBehavior): void {
    const offset = index === 0 ? 0 : this.offsets[index - 1];
    this.viewport.scrollToOffset(offset, behavior);
  }

  private updateContent() {
    const scrollOffset = this.viewport.measureScrollOffset();
    const start = this.offsets.findIndex(o => o > scrollOffset);
    const total = this.offsets.at(-1) ?? 0;
    this.viewport.setTotalContentSize(total);
    this.viewport.setRenderedRange({ start: Math.max(0, start - 2), end:
Math.min(this.itemHeights.length, start + 12) });
  }
}
```

```

    this.viewport.setRenderedContentOffset(start > 0 ? this.offsets[start - 1] : 0);
  }
}

```

The tradeoff: exact precomputed heights give you `O(1) scrollToIndex` for any index (great for "jump to post #57,000") at the cost of needing height data before render (a layout pass on the server, a fixed schema of post "types," or a first-pass estimate that's corrected later). `autosize` needs no upfront data but degrades on jump-to-index and can flicker.

Interviewer intent: Tests whether the candidate can go beyond the built-in `itemSize` input and reason about the `VirtualScrollStrategy` abstraction directly — a strong signal of real CDK depth versus copy-pasted tutorial knowledge.

Scenario 3: Sticky section headers with grouped virtual scroll

Situation: A contacts app groups 50,000 contacts alphabetically (A, B, C...) and needs an alphabet-letter sticky header that stays pinned to the top while its group scrolls past, similar to iOS Contacts.

Question: How do you combine `CdkVirtualScrollViewport` with sticky group headers without breaking virtualization?

Answer: The naive approach — putting a `position: sticky` header as a normal sibling inside `*cdkVirtualFor` — doesn't work well because CSS `position: sticky` needs a stable containing block relative to the *scrolling viewport*, but CDK virtual scroll transforms its content wrapper (`translateY`) for recycling, and headers get virtualized/removed just like rows, so a header can scroll away while its group is still visible, or disappear entirely if it's outside the rendered window.

The standard pattern is to **flatten headers into the same virtualized list as data rows** (so the header is just another "item" with `type: 'header'`), then apply sticky positioning to only the *currently active* header using a separate, non-virtualized overlay element synced to scroll position — not by relying on CSS sticky within the recycled content:

```

type Row = { kind: 'header'; letter: string } | { kind: 'contact'; contact: Contact };

@Component({
  standalone: true,
  imports: [ScrollingModule],
  template: `
    <div class="sticky-header" *ngIf="activeLetter()">{{ activeLetter() }}</div>
    <cdk-virtual-scroll-viewport itemSize="44" class="viewport"
      (scrolledIndexChange)="onScrolledIndex($event)">
      <ng-container *cdkVirtualFor="let row of flatRows(); trackBy: trackByRow">
        <div class="group-header" *ngIf="row.kind === 'header'">{{ row.letter }}</div>
        <div class="contact-row" *ngIf="row.kind === 'contact'">{{ row.contact.name }}</div>
      </ng-container>
    </cdk-virtual-scroll-viewport>
  `,
})
export class ContactListComponent {

```

```

flatRows = signal<Row[]>(buildFlatGroupedRows(contacts));

activeLetter = signal<string>('A');

onScrolledIndex(index: number) {
  // walk backwards from index to find the nearest preceding header row.
  const rows = this.flatRows();
  for (let i = index; i >= 0; i--) {
    if (rows[i].kind === 'header') {
      this.activeLetter.set((rows[i] as any).letter);
      break;
    }
  }
}

trackByRow = (_, row: Row) =>
  row.kind === 'header' ? `h-${row.letter}` : `c-${row.contact.id}`;
}

```

The floating `.sticky-header` div lives *outside* the viewport's virtualized content, positioned absolutely over it, and is only ever updated (not recycled) — so it's immune to CDK's recycling/translation logic. This mirrors how CDK's own `cdk-virtual-scroll-viewport` doesn't natively support sticky content — it deliberately leaves that composition to you, similar to how Material's `mat-table` sticky columns work with `position: sticky` only because the table's own scroll container isn't also doing content-recycling transforms.

An important edge case: `onScrolledIndex` fires on every rendered-range change, so the backward header search should be cheap (it's bounded by group size, not total length) or you should maintain a precomputed "header index for every row index" lookup array for O(1) access instead of scanning.

Interviewer intent: Checks whether the candidate understands that CSS `position: sticky` and CDK's transform-based recycling can conflict, and can design a workaround rather than assuming "just add sticky CSS" works out of the box.

Scenario 4: Scroll-to-item-by-id (deep link to a specific row)

Situation: Users receive a notification link like `/inbox/message/48213` and expect the message list (virtualized, 30,000 items) to open already scrolled to and highlighting that specific message, even though it might be far outside the initially rendered range.

Question: How do you implement reliable "scroll to item by ID" in a virtual-scrolled list?

Answer: `CdkVirtualScrollViewport` exposes `scrollToIndex(index, behavior?)`, but you need the **index**, not the ID, and the target row must exist in the underlying data-bound array at a known position. The approach:

```

import { Component, ElementRef, AfterViewInit, ViewChild, signal, effect } from
  '@angular/core';
import { CdkVirtualScrollViewport, ScrollingModule } from '@angular/cdk/scrolling';
import { ActivatedRoute } from '@angular/router';

@Component({

```

```

standalone: true,
imports: [ScrollingModule],
template: `
  <cdk-virtual-scroll-viewport itemSize="72" class="viewport">
    <div
      *cdkVirtualFor="let msg of messages(); trackBy: trackById"
      [class.highlighted]="msg.id === highlightId()"
      [attr.data-msg-id]="msg.id"
      class="row">
        {{ msg.subject }}
      </div>
    </cdk-virtual-scroll-viewport>
  `
},
})
export class InboxListComponent implements AfterViewInit {
  viewport = viewChild.required(CdkVirtualScrollViewport);
  messages = signal<Message[]>([]);
  highlightId = signal<number | null>(null);

  private route = inject(ActivatedRoute);

  ngAfterViewInit() {
    const targetId = Number(this.route.snapshot.paramMap.get('id'));
    if (!targetId) return;

    const index = this.messages().findIndex(m => m.id === targetId);
    if (index === -1) {
      // Not loaded yet (e.g. lazy-loaded page) – fetch/locate it first, then retry.
      return;
    }

    this.highlightId.set(targetId);
    // scrollToIndex only guarantees the item ends up rendered; to actually
    // center it and confirm, wait a tick for rendering, then verify with
    // renderedRangeStream or scan for the DOM node.
    this.viewport().scrollToIndex(index, 'auto');

    // Two-phase scroll: jump instantly first (no smooth animation over
    // thousands of rows), then optionally nudge with 'smooth' once close,
    // and clear the highlight after a delay.
    setTimeout(() => {
      this.viewport().scrollToIndex(index, 'smooth');
      setTimeout(() => this.highlightId.set(null), 2000);
    }, 50);
  }

  trackById = (_, m: Message) => m.id;
}

```

Important nuances to call out:

- With `cdkFixedSizeVirtualScroll`, `scrollToIndex` is O(1) and instant — the ideal case.
- With `autosize` or a custom strategy, jumping to a far index the viewport has never rendered means the

strategy is *estimating* the offset (average item height × index), so the first jump often lands slightly off, and CDK corrects itself once real heights are measured — this is why the two-phase "jump then smooth-adjust" pattern above exists in production apps (Gmail does something conceptually similar).

- If data is paginated/lazy-loaded from the server and the target ID isn't in the currently loaded page, you need a pre-fetch step: call an API like "give me the page containing message 48213" (often exposed as an index or cursor lookup), splice that page's data in, *then* call `scrollToIndex`.
- Don't call `scrollToIndex` inside `ngOnInit` — the viewport's dimensions aren't measured yet; use `ngAfterViewInit` or, more robustly, subscribe to the viewport's `renderedRangeStream` first emission.

Interviewer intent: Distinguishes candidates who've only used virtual scroll for simple lists from those who've handled the much messier "deep link into a virtualized list" real-world requirement.

Scenario 5: Virtual scroll silently breaking keyboard navigation and screen readers

Situation: QA files an accessibility bug: in a virtualized options list (a custom autocomplete dropdown with 10,000 items), pressing `Arrow Down` repeatedly to move focus works fine for the first ~20 items, then stalls — focus doesn't move further even though the list clearly has more items. Screen reader users also report that JAWS never announces "20 of 10,000", it just says "list, 20 items."

Question: Why does virtual scrolling break keyboard/screen-reader navigation, and how do you fix it?

Answer: Both symptoms have the same root cause: **the DOM only contains the ~20 currently-rendered items**. Native browser focus/arrow-key handling and assistive tech both work off the actual DOM tree, so:

- Arrow-key handlers that do `focusedElement.nextElementSibling.focus()` stop working once the "next sibling" doesn't exist in the DOM yet (it hasn't been virtualized in).
- Screen readers compute list size (`aria-setsize`) from real children by default, so they report only what's rendered, not the logical total.

The fix has three parts:

1. **Manage focus/active-descendant programmatically, not via native DOM focus**, using the roving-tabindex or `aria-activedescendant` pattern driven by a signal for "active index," independent of what's currently rendered:

```
@Component({
  standalone: true,
  imports: [ScrollingModule],
  template: `
    <div role="listbox" [attr.aria-activedescendant]="opt-' + activeIndex()"
      (keydown.arrowdown)="move(1)" (keydown.arrowup)="move(-1)" tabindex="0">
      <cdk-virtual-scroll-viewport itemSize="36" class="viewport">
        <div
          *cdkVirtualFor="let opt of options(); let i = index; trackBy: trackByIndex"
          [id]="opt-' + i"
          role="option"
          [attr.aria-selected]="i === activeIndex()"
          [attr.aria-setsize]="options().length"
          [attr.aria-posinset]="i + 1"
```

```

        class="option">
        {{ opt.label }}
      </div>
    </cdk-virtual-scroll-viewport>
  </div>
},
})
export class VirtualComboboxComponent {
  viewport = viewChild.required(CdkVirtualScrollViewport);
  options = signal<Option[]>([]);
  activeIndex = signal(0);

  move(delta: number) {
    const next = Math.min(Math.max(0, this.activeIndex() + delta), this.options().length - 1);
    this.activeIndex.set(next);
    // Ensure the newly active item is actually scrolled into the rendered range
    // BEFORE assistive tech tries to read it via aria-activedescendant.
    this.viewport().scrollToIndex(next, 'auto');
  }

  trackByIndex = (i: number) => i;
}

```

2. Set `aria-setsize` and `aria-posinset` on every rendered option (as above) so screen readers announce "position 4821 of 10,000" correctly even though only ~20 DOM nodes exist — this is the standard ARIA pattern for virtualized/lazy-loaded collections, explicitly documented in the ARIA Authoring Practices for listbox/grid virtualization.
3. Keep the container's own `role="listbox" / aria-activedescendant` as the focus target — focus never actually moves into the recycled child DOM nodes at all, sidestepping the "focused node got recycled away mid-interaction" problem entirely (which would otherwise throw focus back to `<body>` when CDK detaches that DOM node during recycling).

Interviewer intent: Verifies the candidate knows virtual scroll accessibility is not "solved by CDK automatically" and understands the `aria-activedescendant` + `aria-setsize / aria-posinset` pattern, which is the industry-standard fix, not a hand-wavy one.

Scenario 6: Hybrid virtual scroll with server-side pagination (infinite scroll)

Situation: A search-results feed has 2 million total records server-side; the API only returns 50 at a time via `?page=N`. The product wants seamless infinite scroll that *feels* virtualized (smooth, no visible "Load More" button, scrollbar reflects true total size) without ever holding more than a few hundred records in memory client-side.

Question: How do you architect virtual scroll combined with server-side paging, and what do you do about the scrollbar/scroll-position math when most data doesn't exist client-side yet?

Answer: This requires combining `CdkVirtualScrollViewport`'s **rendered-range notifications** with a data-window/sliding-cache strategy, plus telling the viewport the *true total count* up front (even though you don't have the actual records) so the scrollbar and offset math are correct from the start.

```
import { Component, signal, computed, inject, effect } from '@angular/core';
import { ScrollingModule, CdkVirtualScrollViewport } from '@angular/cdk/scrolling';

@Component({
  standalone: true,
  imports: [ScrollingModule],
  template: `
    <cdk-virtual-scroll-viewport itemSize="64" class="viewport"
      (scrolledIndexChange)="onIndexChange($event)">
      <div *cdkVirtualFor="let row of window(); trackBy: trackByIndex" class="row">
        {{ row?.title ?? 'Loading...' }}
      </div>
    </cdk-virtual-scroll-viewport>
  `,
})
export class SearchResultsComponent {
  private api = inject(SearchApiService);
  readonly PAGE_SIZE = 50;
  readonly totalCount = signal(0); // from an initial count-only API call
  private cache = new Map<number, ResultRow>(); // sparse cache keyed by absolute index
  private loadingPages = new Set<number>();

  // Sparse "window" array of the exact total length; undefined entries render as
  // placeholders.
  window = signal<(ResultRow | undefined)[]>([]);

  constructor() {
    this.api.getCount().subscribe(count => {
      this.totalCount.set(count);
      this.window.set(new Array(count).fill(undefined));
    });
  }

  onIndexChange(index: number) {
    const page = Math.floor(index / this.PAGE_SIZE);
    // Prefetch current + next page so scrolling forward never shows blanks.
    [page, page + 1].forEach(p => this.ensurePageLoaded(p));
  }

  private ensurePageLoaded(page: number) {
    if (this.cache.has(page * this.PAGE_SIZE) || this.loadingPages.has(page)) return;
    this.loadingPages.add(page);
    this.api.getPage(page, this.PAGE_SIZE).subscribe(rows => {
      const arr = [...this.window()];
      rows.forEach((row, i) => {
        const idx = page * this.PAGE_SIZE + i;
        arr[idx] = row;
        this.cache.set(idx, row);
      });
    });
  }
}
```

```

    });
    this.window.set(arr);
    this.loadingPages.delete(page);
    this.evictFarPages(page);
  });
}

private evictFarPages(currentPage: number) {
  // Keep only a bounded window in memory (e.g. 10 pages either side) so the
  // sparse array doesn't quietly hold millions of resolved rows forever.
  const arr = [...this.window()];
  for (const [idx] of this.cache) {
    const p = Math.floor(idx / this.PAGE_SIZE);
    if (Math.abs(p - currentPage) > 10) {
      arr[idx] = undefined;
      this.cache.delete(idx);
    }
  }
  this.window.set(arr);
}

trackByIndex = (i: number) => i;
}

```

Key design decisions to articulate:

- The array passed to `*cdkvirtualFor` must be the **full logical length** (`totalCount`, possibly 2 million) with `undefined` placeholders — this is what makes the scrollbar thumb size/position accurate immediately, rather than growing as more pages load (the jarring "shrinking scrollbar" effect users hate in naive infinite scroll).
- `scrolledIndexChange` (or `renderedRangeStream` for the full start/end range) drives *prefetching*, not rendering directly — always fetch one page ahead of the scroll direction to hide network latency.
- Eviction is mandatory for a 2M-row dataset — without it you're back to holding a huge object graph in memory, just built incrementally instead of all at once. Bound the cache to a sliding window around the current scroll position.
- Placeholder rows ("Loading...") should have the *same* height as real rows or you break the fixed-size assumption and get jumping.
- This is fundamentally different from classic "infinite scroll" (which just appends to a growing array and re-renders everything) — real virtual+paginated hybrid never grows the *rendered* DOM, and bounds the *cached* data too.

Interviewer intent: Tests system-design thinking for a case where naive "virtual scroll" and naive "infinite scroll" tutorials both fail at scale — a strong signal for senior/staff-level candidates.

Scenario 7: Justifying virtual scroll vs. pagination to a skeptical PM

Situation: A PM pushes back on adding CDK virtual scroll complexity and asks, "why not just paginate 50 rows per page with Next/Previous buttons? It's simpler and we already have the API supporting `?page=N`."

Question: What's your technical comparison of virtual scroll vs. pagination, and when would you actually recommend pagination instead?

Answer: This is a legitimate tradeoff, not a strictly-superior-strategy question, and a good answer weighs UX, engineering cost, and specific failure modes on both sides:

Virtual scroll advantages:

- Continuous scanning UX — users exploring/searching large datasets (log viewers, chat history, spreadsheets) strongly prefer uninterrupted scroll over clicking Next repeatedly; task time and error rate are measurably worse with pagination for "find the item I'm looking for" tasks.
- No layout thrash of full page reloads/pagination boundary jumps — scroll position is continuous and predictable.
- Handles "jump near a specific position" (e.g., a timeline scrubber) more naturally than "compute which page number contains item X."

Pagination advantages (often overlooked):

- **Deep-linkable, bookmarkable, SEO-friendly** state — `?page=4` is trivial to share/cache/index; a scroll offset into a virtualized list isn't naturally a URL, and building "scroll to index" deep-linking (Scenario 4) is nontrivial extra engineering.
- **Simpler mental model & lower total engineering cost** — no `VirtualScrollStrategy` internals, no sticky-header workarounds, no accessibility retrofitting (Scenario 5), no nested-scroll-container gotchas (Scenario 8). For a 5,000-row admin table used by 10 internal users, pagination is almost always the right call — virtual scroll is solving a performance problem you may not have.
- **More forgiving of variable-height/complex content** — pagination doesn't need offset math since each page is a fresh, small, boundedly-sized render.
- **Easier to reason about analytics/testing** — "page 3 was viewed" is a clean event; "user scrolled to row 48,213" is not.

Actual performance comparison: with a fixed-size DOM budget (say, render ~30 rows at a time either way), *rendering* cost is similar between "render page 3 of 50-row pages" and "render the currently visible virtual window" — the win from virtual scroll isn't magic, it's specifically that the *scrollable container* behaves as if all N rows exist (continuous scrollbar, no reload boundary) while never mounting more than ~30-60 DOM nodes. The recommendation: use pagination by default for internal/admin tools and moderate datasets (under a few thousand rows, or where deep-linking matters); reach for virtual scroll specifically when (a) the data is naturally continuous/scannable (feeds, chat, logs, spreadsheets) and (b) row counts are large enough (tens of thousands+) that even paginated 50-row chunks feel like an artificial, annoying interruption to the user's actual task.

Interviewer intent: Filters out candidates who reflexively reach for virtual scroll as a resume-driven-development default rather than making a reasoned engineering tradeoff — senior interviewers specifically listen for "it depends" with concrete criteria.

Scenario 8: Virtual scroll inside a modal breaks — page scrolls instead of the list

Situation: A "Select Product" modal contains a `cdk-virtual-scroll-viewport` listing 20,000 SKUs. On desktop it works, but on the QA's laptop with a trackpad, scrolling over the list sometimes scrolls the *background page* behind the modal instead of the list, and occasionally the whole modal janks/flickers. There's also a report that opening the modal a second time shows the list starting from the previous scroll position instead of the top.

Question: What's causing the scroll bleed-through and stale scroll position, and how do you fix nested virtual scroll inside overlay components?

Answer: Three distinct bugs are being conflated here, each with a different root cause:

1. Scroll bleed-through to the background page. This happens when the modal backdrop/body doesn't have scroll containment, so at the top/bottom edge of the virtual viewport's scroll range, the wheel/touch event's remaining delta "overscrolls" into the parent scrollable ancestor (the page body). The fix is CSS overscroll containment on the viewport, not anything virtual-scroll-specific:

```
cdk-virtual-scroll-viewport {  
  overscroll-behavior: contain; /* stops scroll chaining to parent once at boundary */  
}
```

Additionally, the CDK Overlay (which backs `CdkDialog` / Angular Material `MatDialog`) already applies `cdk-global-scrollblock` to `<html>` while a modal is open specifically to prevent background scroll — if this is bypassed (e.g., a custom modal not built on `CdkDialog` / `Overlay`), that's the real bug: **build the modal on `@angular/cdk/dialog` or `@angular/cdk/overlay`**, don't hand-roll a modal `div` with manual `position: fixed`, since you lose this scroll-blocking behavior along with focus trapping.

2. Modal-open jank/flicker. Frequently caused by the virtual scroll viewport being measured (`ngAfterViewInit` / `ResizeObserver`) while the modal's own open animation is still transitioning its size/opacity — CDK virtual scroll measures the viewport's `clientHeight` to compute the rendered range, and if that measurement happens mid-transition (e.g., height animating from 0), it renders the wrong number of items, then has to correct itself once the transition ends, causing a visible re-layout. Fix: don't animate the *height* of the container that wraps the viewport; animate opacity/transform only, or call `viewport.checkViewportSize()` after the open transition's `transitionend` / `animationend` fires.

3. Stale scroll position on reopen. If the modal component instance is retained (e.g., kept in the DOM with `*ngIf="open()"` toggling, or the component isn't destroyed/recreated between opens because it's a singleton service-driven overlay), the `CdkVirtualScrollViewport`'s internal scroll offset persists. If the desired behavior is "always start at top," explicitly reset it when the modal opens:

```
export class ProductPickerModalComponent {
  viewport = viewChild.required(CdkVirtualScrollViewport);

  onModalOpened() {
    // Reset to top; also good practice to re-measure in case content changed
    // while the modal was closed (e.g. product list refreshed).
    this.viewport().scrollToIndex(0);
    this.viewport().checkViewportSize();
  }
}
```

If instead the *desired* UX is "remember where I was" (common for a picker the user reopens repeatedly), store the last scroll index in a service/signal and restore it explicitly with `scrollToIndex` in `onModalOpened` rather than relying on incidental DOM retention, which is fragile and breaks the moment someone changes the modal to be recreated per-open.

Interviewer intent: Tests whether the candidate can debug composed-component interactions (overlay + virtual scroll + animation) rather than treating virtual scroll as an isolated black box — a very common real production bug class.

Scenario 9: Horizontal virtual scroll for a Gantt-chart-style timeline

Situation: A project-planning tool needs a horizontally-scrolling timeline with thousands of day-columns, each ~80px wide, and vertically a list of ~500 task rows — effectively a 2D virtualization problem (only render the visible day-range × visible task-range).

Question: Does CDK virtual scroll support horizontal scrolling and 2D virtualization out of the box? How would you architect this?

Answer: `CdkVirtualScrollViewport` supports a single scroll axis via the `orientation` input (`"vertical"` default, or `"horizontal"`), so horizontal-only virtualization is directly supported:

```
<cdk-virtual-scroll-viewport orientation="horizontal" itemSize="80" class="timeline-viewport">
  <div *cdkVirtualFor="let day of days(); trackBy: trackByDay" class="day-column">
    {{ day.label }}
  </div>
</cdk-virtual-scroll-viewport>
```

True 2D virtualization (both axes simultaneously) is not built into CDK — there's no first-class `CdkVirtualScrollViewport` variant that virtualizes rows and columns together. For a Gantt chart you need one of:

1. **Outer vertical virtual scroll for rows, inner rendering logic (not a nested viewport) for the horizontal window** — the outer `cdk-virtual-scroll-viewport` (vertical) virtualizes the ~500 task rows normally. Each rendered row component independently computes which day-columns are visible using the *shared* horizontal scroll position (kept in a single signal/service, not per-row scroll containers, since a Gantt chart needs all rows to scroll horizontally in lockstep):

```

@Injectables({ providedIn: 'root' })
export class TimelineScrollState {
  scrollLeft = signal(0);
  visibleDayRange = computed(() => {
    const left = this.scrollLeft();
    const startDay = Math.floor(left / 80);
    const visibleCount = Math.ceil(window.innerWidth / 80) + 4; // + buffer
    return { start: startDay, end: startDay + visibleCount };
  });
}

@Component({
  selector: 'app-gantt-row',
  standalone: true,
  template: `
    <div class="row" [style.width.px]="totalDays() * 80">
      @for (day of visibleDaysSlice(); track day.index) {
        <div class="cell" [style.transform]="`translateX(${(day.index * 80) + 'px'})`">
          <app-task-bar-segment [task]="task" [day]="day" />
        </div>
      }
    </div>
  `,
})
export class GanttRowComponent {
  scrollState = inject(TimelineScrollState);
  visibleDaysSlice = computed(() => {
    const { start, end } = this.scrollState.visibleDayRange();
    return this.allDays.slice(start, end);
  });
}

```

One real `<div>` with `overflow: hidden` (or just clipped by the outer container) carries the horizontal scroll listener that updates `TimelineScrollState.scrollLeft`; all row components read from the same signal so they stay in sync without each managing its own scroll listener.

2. **Or, if row count is small but day-range is enormous** (e.g., a 10-year timeline), flip it: use `CdkVirtualScrollViewport` with `orientation="horizontal"` as the *outer* virtualizer for columns, and render full (non-virtualized) row content for the currently-visible day slice.
3. **For genuinely large-in-both-dimensions grids** (spreadsheet-scale, 100k rows × 1000 columns), reach for a purpose-built 2D virtualization library or write a fully custom `VirtualScrollStrategy`-inspired grid renderer computing both a visible row range and visible column range from two independent scroll offsets — CDK's abstraction genuinely doesn't extend cleanly to this case, and forcing nested `cdk-virtual-scroll-viewport`s (vertical outer, horizontal inner per row) causes scroll-listener storms (500 independent scroll listeners) and desynchronized horizontal positions between rows, which is a common failed first attempt.

Interviewer intent: Reveals whether the candidate knows the actual boundaries of what CDK's viewport handles (1D) versus what requires custom architecture (2D) — important for teams building data-grid-like products.

Scenario 10: Images loading late change item height, causing scroll jump

Situation: A photo gallery list uses `autosize` virtual scroll. Each row shows a thumbnail that loads asynchronously (lazy `<img>` with `loading="lazy"`), and once the image finishes loading, the row's height changes from a 40px skeleton placeholder to 200px. Users report the whole list "jumps" — content they were reading shifts up or down unexpectedly as images above the viewport load in.

Question: Why does this happen with `autosize`, and how do you prevent layout shift when item content resizes asynchronously?

Answer: `CdkAutoSizeVirtualScroll` estimates item sizes by measuring the DOM *after* it renders, then corrects its internal offset table. If an item *above* the current viewport resizes after the user has already scrolled past it (image finishes loading late), the strategy recalculates that item's contribution to total scroll height and adjusts — which, from the user's point of view, is exactly a layout shift/jump, conceptually identical to Cumulative Layout Shift (CLS) issues in regular web pages, just inside a virtualized container.

The fix is to **eliminate the async height change entirely**, not to fight the virtual-scroll strategy after the fact:

```
@Component({
  selector: 'app-photo-row',
  standalone: true,
  template: `
    <div class="photo-row" [style.aspect-ratio]="photo.width + ' / ' + photo.height">
      <img
        [src]="photo.thumbnailUrl"
        [width]="photo.width"
        [height]="photo.height"
        loading="lazy"
        (load)="loaded.set(true)"
        [class.loaded]="loaded()" />
      @if (!loaded()) {
        <div class="skeleton"></div>
      }
    </div>
  `,
  styles: [
    .photo-row { position: relative; width: 100%; }
    img { width: 100%; height: 100%; object-fit: cover; opacity: 0; transition: opacity .2s; }
  ],
  export class PhotoRowComponent {
    @Input({ required: true }) photo!: Photo;
    loaded = signal(false);
  }
```

The critical technique: **reserve the final size up front** using the known `width/height` (or `aspect-ratio`) metadata that the API already returns alongside the thumbnail URL (most image/photo APIs — S3-backed CMSs, Cloudinary, etc. — return dimensions in the same payload as the URL). The skeleton is absolutely positioned *inside* an already-correctly-sized container, so when the image swaps in via opacity fade, **the container never resizes** — only its visual content does. This means the `autosize` (or even fixed-size, if all photos share the same aspect box) strategy sees a stable height from the very first paint, and there's no post-load layout shift to correct.

If height truly can't be known ahead of time (no metadata from the API), the fallback is to accept `autosize`'s correction behavior but minimize its visible impact: only allow height corrections for items *below* the current scroll position (already effectively how `autosize` behaves, since it only measures rendered items, but you can additionally choose to lazy-load images only once they're near-visible via `IntersectionObserver`, rather than lazy-loading everything including far-off items, so height corrections don't cascade across dozens of rows at once during a fast scroll/flip).

Interviewer intent: Connects the general web-perf concept of Cumulative Layout Shift to the specific mechanics of `autosize`, testing whether the candidate treats virtual scroll bugs as isolated CDK quirks or recognizes them as instances of familiar broader problems.

Scenario 11: Search/filter resets scroll position unexpectedly

Situation: A virtualized 5,000-item list has a live search box. Typing a search term filters the underlying array (via a `computed()` signal) and the list correctly shows fewer results — but users complain that after clearing the search box, the list is scrolled back to the top instead of returning to where they were before searching, and also that mid-typing, the list flickers to the top on every keystroke.

Question: How do you manage scroll position correctly across filtering, and why does it reset on every keystroke here?

Answer: Two separate issues:

Flicker to top on every keystroke: When the bound array reference changes (a new filtered array is produced by `computed()` on every keystroke) and the array is *shorter* than before, `CdkVirtualScrollViewport`'s current scroll offset may now point past the end of the new, shorter content, so CDK clamps the scroll position back into range — visually this looks like "jump to top" if the previous offset was now out of bounds, or worse, if `trackBy` isn't properly matching items across the two arrays, Angular treats every remaining item as new (destroy + recreate), which itself resets any virtual scroll internal assumptions about rendered content:


```

@Component({ /* ... */ })
export class SearchableListComponent {
  private allItems = signal<Item[]>([]);
  searchTerm = signal('');

  filtered = computed(() => {
    const term = this.searchTerm().toLowerCase();
    return term ? this.allItems().filter(i => i.name.toLowerCase().includes(term)) :
this.allItems();
  });

  trackById = (_, item: Item) => item.id; // MUST be stable across filter changes
}

```

`trackBy` alone doesn't prevent the "shorter array clamps scroll" behavior — that's inherent and arguably correct (you can't stay scrolled to position 4000 in a 50-item filtered result). The real fix is to make the *product* decision explicit and implement it deliberately rather than accept whatever the clamp produces:

```

export class SearchableListComponent {
  viewport = viewChild.required(CdkVirtualScrollViewport);
  private preSearchScrollIndex = 0;
  private wasSearching = false;

  onSearchTermChange(term: string) {
    const isSearching = term.length > 0;
    if (isSearching && !this.wasSearching) {
      // Entering search mode: remember where the user was, then jump to top
      // deliberately (searching implies "start scanning results from the top").
      this.preSearchScrollIndex = this.viewport().measureScrollOffset() / 48; // itemSize
      this.viewport().scrollToIndex(0);
    } else if (!isSearching && this.wasSearching) {
      // Search cleared: restore prior position instead of leaving it at top.
      this.viewport().scrollToIndex(Math.round(this.preSearchScrollIndex));
    }
    this.wasSearching = isSearching;
    this.searchTerm.set(term);
  }
}

```

Debounce to avoid thrash: filtering on *every* keystroke against 5,000 items with `.includes()` is cheap enough computationally, but the *scroll-state churn* (rendered range recalculating, `trackBy` diffing) on every keystroke is what causes visible flicker under fast typing. Standard fix: debounce the signal driving `filtered` (e.g., via `toSignal(searchTerm$.pipe(debounceTime(150)))` or a manual `setTimeout`-based debounce updating a separate `debouncedTerm` signal), so recomputation and the associated scroll-position handling only happen once typing pauses, not on every character.

Interviewer intent: Explores whether the candidate treats scroll position as meaningful application state that needs explicit product/UX decisions (not just an implementation detail CDK "handles"), and understands the interaction between signal-driven re-filtering and `trackBy`.

Scenario 12: Drag-and-drop reordering inside a virtualized list

Situation: A task-board list of 10,000 tasks needs drag-and-drop reordering (using `@angular/cdk/drag-drop`), but the team discovers `cdk-virtual-scroll-viewport` and `cdkDropList` don't compose well: dragging an item near the top/bottom edge doesn't auto-scroll the virtualized viewport, and dropping an item sometimes reorders the wrong underlying data index because the *rendered* DOM order doesn't match the *logical* array order during a drag.

Question: What are the specific failure modes of combining CDK drag-drop with CDK virtual scroll, and how do you resolve them?

Answer: These are two separate, well-known integration gaps:

1. No auto-scroll near viewport edges during drag. `cdkDropList`'s auto-scroll feature (`cdkDropListAutoScrollDisabled`) is designed around normal scrollable containers where all sibling elements exist in the DOM; because `cdk-virtual-scroll-viewport` only renders a subset, CDK drag-drop's auto-scroll detection can behave inconsistently (it does generally work since Angular CDK added explicit support for recognizing `CdkVirtualScrollViewport` as a scrollable ancestor, but teams on older CDK versions or with custom scroll containers frequently hit this). The robust fix is to drive scrolling explicitly from the drag events rather than relying entirely on the built-in heuristic:

```
@Component({ /* ... */ })
export class TaskBoardComponent {
  viewport = viewChild.required(CdkVirtualScrollViewport);

  onDragMoved(event: CdkDragMove) {
    const viewportEl = this.viewport().elementRef.nativeElement;
    const rect = viewportEl.getBoundingClientRect();
    const pointerY = event.pointerPosition.y;
    const edgeThreshold = 60;

    if (pointerY < rect.top + edgeThreshold) {
      this.viewport().scrollToOffset(this.viewport().measureScrollOffset() - 15);
    } else if (pointerY > rect.bottom - edgeThreshold) {
      this.viewport().scrollToOffset(this.viewport().measureScrollOffset() + 15);
    }
  }
}
```

2. Reordering the wrong index because DOM order $\neq$ logical order. This is the more dangerous bug: `moveItemInArray` (from `@angular/cdk/drag-drop`) uses the indices reported by `cdkDropList`'s drag events, which are based on the *rendered* children's position among currently-mounted DOM nodes — but with virtual scroll, the currently-mounted nodes are a *window* into the full array (e.g., DOM positions 0-19 might correspond to logical array indices 340-359 after scrolling). If you naively do `moveItemInArray(this.tasks(), event.previousIndex, event.currentIndex)` using the raw event indices, you silently corrupt the array by moving the wrong items.

The fix is to translate rendered/DOM-relative indices into true array indices using the viewport's *rendered range offset* before mutating the source array:

```
onDrop(event: CdkDragDrop<Task[]>) {
  const range = this.viewport().getRenderedRange(); // { start, end }
  const trueFrom = range.start + event.previousIndex;
  const trueTo = range.start + event.currentIndex;

  const updated = [...this.tasks()];
  moveItemInArray(updated, trueFrom, trueTo);
  this.tasks.set(updated);
}
```

In practice, many teams sidestep both issues by **disabling drag-and-drop reordering entirely while the list is virtualized**, restricting reordering to a non-virtualized "compact" view, or implementing reordering via explicit up/down buttons or a modal "move to position" input instead of freeform drag — because the DOM-index-vs-logical-index translation is a genuine correctness hazard that's easy to get subtly wrong (off-by-one bugs here silently reorder the wrong tasks, which is a data-integrity bug, not just a visual glitch), and worth calling out as a legitimate design alternative in the interview answer.

Interviewer intent: Tests awareness that composing two different CDK modules (`scrolling` + `drag-drop`) isn't automatically safe, and specifically whether the candidate recognizes the DOM-index vs. data-index mismatch as a correctness bug, not merely a UX rough edge.

Scenario 13: Tuning `minBufferPx` / `maxBufferPx` to fix white flashes during fast scroll (fling)

Situation: On a fast trackpad "fling" scroll gesture through a virtualized 50,000-row list, users briefly see blank white rows before content pops in, especially noticeable on lower-end laptops. Reducing row complexity didn't fully fix it.

Question: What causes blank flashes during fast scrolling in CDK virtual scroll, and how do `minBufferPx` / `maxBufferPx` help?

Answer: `cdk-virtual-scroll-viewport` renders a buffer of extra items just outside the visible viewport in both directions, controlled by two inputs: `minBufferPx` (the minimum amount of buffer, in pixels, to always keep rendered beyond the visible area) and `maxBufferPx` (the buffer size the viewport renders up to when it needs to add more, batched to avoid rendering one row at a time). Defaults are modest (100px/200px) because they trade off memory/render cost against fling-scroll smoothness.

During a fast fling, the scroll position can move hundreds or thousands of pixels within a single frame (or a few frames) — if the buffer isn't large enough to cover that jump, the browser paints the newly-visible area before Angular has finished creating/patching the DOM nodes for it, producing a visible blank flash until the next change-detection cycle catches up.

```

<cdk-virtual-scroll-viewport
  itemSize="48"
  minBufferPx="600"
  maxBufferPx="1200"
  class="viewport">
  <div *cdkVirtualFor="let row of rows(); trackBy: trackById" class="row">{{ row.label }}
</div>
</cdk-virtual-scroll-viewport>

```

Guidance on tuning:

- Increase `minBufferPx` to roughly cover the *maximum* fling distance you want to guarantee is pre-rendered — e.g., if a hard fling moves ~800px in the time it takes Angular to render a batch, set `minBufferPx` to that magnitude, not just "a bit more than the default."
- `maxBufferPx` should be a couple hundred pixels above `minBufferPx` — it governs batch size when *adding* new buffer content (rendering more at once reduces per-frame render calls but increases jank if a huge batch is created synchronously in one frame; too small a gap means many small renders back-to-back, which has its own overhead).
- **This is a tradeoff against baseline memory/CPU**, not a free win — a larger buffer means more permanently-mounted DOM nodes and bindings even when the user isn't flinging, which raises the steady-state cost. For 100k+ rows with heavy row templates (charts, rich cell renderers), aggressively large buffers can reintroduce the very jank virtual scroll was meant to fix.
- If tuning buffers doesn't fully resolve the flash, the next lever is **reducing per-row render cost** (simpler templates, `OnPush` /signal-based components so recycled rows patch cheaply, avoiding expensive pipes recalculating per row) so that even the buffer-render batches complete within a frame budget (~16ms) rather than blocking.
- As a last resort for very heavy rows, consider rendering a lightweight placeholder/skeleton immediately and hydrating full row content asynchronously (similar to the image lazy-load pattern in Scenario 10), decoupling "row exists in DOM for scroll math" from "row's expensive content is fully rendered."

Interviewer intent: Confirms the candidate has actually tuned these specific CDK inputs in a real project rather than just knowing `itemSize` exists — a good filter for "read the docs once" versus "shipped this."

Scenario 14: Form inputs inside virtualized rows lose focus/state on recycling

Situation: An editable spreadsheet-like grid virtualizes 20,000 rows, each containing a few `<input>` fields bound with `[(ngModel)]` /reactive form controls per row. Users report that while typing into a cell near the edge of the viewport and scrolling slightly, focus jumps away and, worse, occasionally a value they typed appears in a *different* row after scrolling back.

Question: Why does recycling cause data to "leak" between rows, and how do you build editable virtualized rows safely?

Answer: This is the most dangerous class of virtual scroll bug because it corrupts user data, not just visuals. The root cause: CDK virtual scroll **recycles DOM nodes and component instances**, not values — when a row scrolls out of the rendered range and a new row scrolls in, Angular may reuse the same component instance/DOM subtree and simply rebind it to new `@Input()` data. If a form control's value is being tracked by *internal component state that isn't correctly reset from the new input* (a very common bug: a component caches an input's initial value in `ngOnInit` into a local mutable field, or a reactive form's `FormControl` is created once and never updated when the row's underlying data model changes), the old row's typed value visually "sticks" to the recycled DOM node and appears attached to the new row's data.

The safe pattern is: **never let a recycled row component own long-lived, independently-mutable state that isn't synchronized from its `@Input()` /model on every change** — treat each row as a pure projection of its bound data plus explicit, traceable edit events:

```
@Component({
  selector: 'app-editable-row',
  standalone: true,
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <input
      [value]="row().value"
      (input)="onInput($event)"
      [attr.data-row-id]="row().id" />
  `,
})
export class EditableRowComponent {
  row = input.required<RowData>();
  edited = output<{ id: string; value: string }>();

  onInput(event: Event) {
    const value = (event.target as HTMLInputElement).value;
    // Emit immediately to the parent's authoritative data store – do not
    // keep a local "draft" signal/FormControl that could persist across
    // recycling if this same component instance gets rebound to a new row.
    this.edited.emit({ id: this.row().id, value });
  }
}
```

Two additional required guardrails:

1. **`trackBy` must key on the row's stable ID, never on array index.** If `trackBy` returns the index, Angular considers "row at position 5" the same logical item across data changes even when the underlying row's ID actually changed (e.g., after a sort/filter), which is precisely the mechanism that causes state to appear "glued" to a DOM position rather than a data identity.
2. **Uncontrolled `<input>` value must be driven by `[value]` binding, not just set once.** Using `[(ngModel)]` backed by a `FormControl` created in the row component's constructor and never updated when `row()` changes is the classic bug — the fix is either to rebuild the `FormControl`'s value reactively via an `effect()` watching `row()`, or (simpler, as shown above) skip Reactive Forms for the per-cell value entirely and use a plain `[value] / (input)` pattern with the parent as the single source of truth, re-synchronizing on every recycle since `[value]` is re-evaluated on every change detection pass regardless of whether the DOM node is "new" or recycled.

The broader principle to state explicitly in the interview: **with virtual scroll, "component instance" and "logical row" are not the same thing** — any state you'd normally consider "owned by this row's component" must instead be treated as ephemeral/derived, with the true source of truth living in the parent's data array (or a form array indexed by row ID, not position).

Interviewer intent: This is the single most important virtual-scroll correctness question for any editable-grid scenario — tests whether the candidate understands component recycling deeply enough to prevent data corruption, the most severe possible bug class here.

Scenario 15: Unit and E2E testing a virtualized component

Situation: A QA engineer complains that Cypress/Playwright tests for a virtualized order list keep failing intermittently with "element not found" when trying to click on order #4,832 — the test scrolls the container and immediately tries to click, but the row sometimes isn't in the DOM yet. A separate Jasmine/Jest unit test that asserts "all 500 items are in the DOM" also fails against the real implementation.

Question: How do you properly write unit and E2E tests against components using `CdkVirtualScrollViewport`?

Answer: Both failures stem from the same fact the tests need to account for: **CDK virtual scroll asynchronously updates the rendered DOM range after scroll events**, via `NgZone`-scheduled work — there's an inherent delay between "scroll happened" and "target row exists in DOM," and the *total* rendered item count is never all 500 at once by design.

Unit tests (TestBed): Don't assert on total DOM count; instead, either test the *data layer* (filtering/sorting logic) independently of rendering, or explicitly drive the viewport and wait for stability:

```
it('renders the item at a scrolled-to index', fakeAsync(() => {
  const fixture = TestBed.createComponent(OrderListComponent);
  fixture.detectChanges();
  const viewport = fixture.debugElement.query(By.directive(CdkVirtualScrollViewport))
    .injector.get(CdkVirtualScrollViewport);

  viewport.scrollToIndex(300);
  viewport.checkViewportSize();
  fixture.detectChanges();
  tick(); // flush the scheduled render-range update
  fixture.detectChanges();

  const rendered = fixture.nativeElement.querySelectorAll('.order-row');
  expect(Array.from(rendered).some((el: any) => el.textContent.includes('order
#301'))).toBeTrue();
}));
```

Key technique: call `tick()` (with `fakeAsync`) or `await fixture.whenStable()` after `scrollToIndex`, since the rendered-range recalculation is scheduled asynchronously, not synchronous with the call.

E2E tests (Cypress/Playwright): Never assume a virtualized item exists in the DOM just because it's "in the data." The robust pattern is scroll-and-poll:

```
// Playwright example
```

```

async function scrollUntilVisible(page: Page, testId: string, maxAttempts = 30) {
  const viewport = page.locator('cdk-virtual-scroll-viewport');
  for (let i = 0; i < maxAttempts; i++) {
    const locator = page.locator(`[data-testid="${testId}"]`);
    if (await locator.count() > 0) {
      await locator.scrollIntoViewIfNeeded();
      return locator;
    }
    await viewport.evaluate(e1 => e1.scrollBy(0, 600));
    await page.waitForTimeout(50);
  }
  throw new Error(`Item ${testId} never rendered after scrolling`);
}

```

Even better, if the app exposes a "scroll to ID" affordance already (Scenario 4), tests should drive *that* application-level API (e.g., navigate to a URL that deep-links to the order) rather than simulating raw scroll-and-poll, since it tests the same code path real users exercise and is far less flaky than guessing scroll distances.

Also worth mentioning: visual-regression/snapshot tests against virtualized lists are inherently less deterministic (exact rendered set depends on viewport pixel height, which can vary by test-runner environment/OS font rendering), so snapshot assertions should target a fixed, explicit scroll position and a fixed-size iframe/viewport, and ideally assert on specific element content/attributes rather than full-page pixel diffs.

Interviewer intent: Assesses whether the candidate has actually had to make virtual-scroll UI testable/CI-stable, a very common practical pain point that "demo-only" CDK knowledge doesn't surface.

Scenario 16: CSS enter/leave animations conflict with row recycling

Situation: Product wants each new chat message to fade/slide in when it appears (`@angular/animations` or CSS transitions triggered by `*ngIf` /class toggling). Once virtual scroll is added to the chat log (to handle 50,000-message history), the animations start firing on *every* row whenever the user scrolls — messages that have existed for hours "slide in" again as if new, because scrolling causes them to re-enter the rendered range.

Question: Why do enter animations misfire with virtual scroll, and how do you scope animations to genuinely new content only?

Answer: The animation trigger is almost certainly keyed off the component's structural (re)creation — Angular's `:enter / :leave` animation states fire whenever a view is inserted into the DOM, and with virtual scroll, a row scrolling back into the rendered range **is** a fresh view insertion (either a newly created component instance, or a recycled instance being re-attached), which the animation system can't distinguish from "this message was just received by the user." The component has no way to know "was I just created because of new data, or just because I scrolled into view again."

The fix is to **stop keying the animation off DOM insertion and key it off actual data novelty**, tracked explicitly by the application:

```

@Component({
  selector: 'app-chat-message',
  standalone: true,

```



```

template: `
  <div class="message" [class.animate-in]="isNewlyArrived()">
    {{ message().text }}
  </div>
`,
})
export class ChatMessageComponent {
  message = input.required<ChatMessage>();
  private seenIds = inject(SEEN_MESSAGE_IDS); // shared Set<string>, app-lifetime

  isNewlyArrived = computed(() => {
    const id = this.message().id;
    const isNew = !this.seenIds.has(id);
    if (isNew) {
      // Mark as seen immediately so re-render/recycling never re-triggers it,
      // regardless of how many times this ID scrolls in/out of view.
      queueMicrotask(() => this.seenIds.add(id));
    }
    return isNew;
  });
}

```

```

// Provided at the chat feature level, persists across scroll/recycling,
// reset only when the whole chat session/conversation actually changes.
export const SEEN_MESSAGE_IDS = new InjectionToken<Set<string>>('seenMessageIds', {
  factory: () => new Set<string>(),
});

```

With this, `animate-in` (a CSS class driving a `@keyframes` /transition) is applied only the very first time a given message ID is ever rendered anywhere in the app's lifetime for this session — scrolling that message out and back in later never re-adds the class, because its ID is already recorded in `seenIds`, independent of whether the component/DOM node instance is fresh or recycled.

A subtlety worth mentioning: if using Angular's `@angular/animations` package with `[@fadeIn]` triggers bound to a boolean, the same principle applies — bind the trigger's state to `isNewlyArrived()` (computed from a persistent "seen" set), not to `*ngIf` /structural presence, since structural presence is exactly what virtual scroll churns constantly by design.

Interviewer intent: Tests whether the candidate recognizes that Angular's animation system operates on DOM lifecycle events, which virtual scroll deliberately manipulates for recycling — a subtle interaction that only surfaces once animations and virtualization are combined in the same feature.

Scenario 17: Multi-column responsive grid (e.g., photo grid) with virtual scroll

Situation: A photo gallery displays items in a responsive CSS grid (`grid-template-columns: repeat(auto-fill, minmax(200px, 1fr))`), showing anywhere from 3 to 8 columns depending on viewport width. The team wants to virtualize 30,000 photos, but `cdk-virtual-scroll-viewport` with `*cdkVirtualFor` naturally treats each item as its own row — directly wrapping the grid doesn't virtualize correctly because CDK doesn't

know multiple items share a row.

Question: How do you virtualize a multi-column grid layout with CDK, given that `*cdkVirtualFor` is fundamentally a single-column (one item per virtualized row) abstraction?

Answer: Correct instinct — `CdkVirtualScrollViewport / CdkVirtualForOf` virtualizes a **flat list of "rows"**, where each row's height is what `itemSize` (or `autosize`) tracks; it has no native concept of "N items packed horizontally per row." The standard solution is to **virtualize at the row level, not the item level** — chunk the flat photo array into fixed-size groups (based on however many columns currently fit) and treat each *group* as one virtualized item:

```
import { Component, signal, computed, HostListener } from '@angular/core';
import { ScrollingModule } from '@angular/cdk/scrolling';

@Component({
  standalone: true,
  imports: [ScrollingModule],
  template: `
    <cdk-virtual-scroll-viewport [itemSize]="rowHeight()" class="viewport">
      <div *cdkVirtualFor="let group of photoRows(); trackBy: trackByGroup" class="photo-
row"
        [style.grid-template-columns]="`repeat(' + columns() + ', 1fr)`${>
          @for (photo of group; track photo.id) {
            <img [src]="photo.thumbUrl" class="thumb" />
          }
        </div>
      </cdk-virtual-scroll-viewport>
    `,
  styles: [`.photo-row { display: grid; gap: 8px; }`],
})
export class PhotoGridComponent {
  private allPhotos = signal<Photo[]>([]);
  columns = signal<number>(this.computeColumns());
  rowHeight = signal(220);

  photoRows = computed(() => {
    const cols = this.columns();
    const photos = this.allPhotos();
    const rows: Photo[][] = [];
    for (let i = 0; i < photos.length; i += cols) {
      rows.push(photos.slice(i, i + cols));
    }
    return rows;
  });

  @HostListener('window:resize')
  onResize() {
    const newCols = this.computeColumns();
    if (newCols !== this.columns()) this.columns.set(newCols);
  }

  private computeColumns(): number {
    const width = window.innerWidth;
  }
}
```

```

    if (width > 1400) return 6;
    if (width > 1000) return 4;
    if (width > 600) return 3;
    return 2;
  }

  trackByGroup = (_, number, group: Photo[]) => group[0]?.id ?? _;
}

```

Important implementation details:

- `columns()` recalculating on resize **must** trigger `photoRows()` to re-chunk — since it's a `computed()` deriving from both `allPhotos` and `columns`, this happens automatically; but note that changing column count re-derives entirely new row groupings, which means `trackByGroup` (keyed on the first photo's ID) will see almost all "rows" as new after a resize — this is expected and acceptable since a resize is a rare, deliberate layout change, not a per-scroll-frame operation.
- `itemSize / rowHeight` must reflect the actual row height, which depends on the *aspect ratio and column width* of the images — if photos aren't uniform, you're back to the variable-height problem (Scenario 2) but at the row-group level instead of the item level; `autosize` still works here since it measures whole rows.
- This pattern (group flat data into fixed-size chunks, virtualize the chunks) is the general technique for **any grid-of-items virtualization with CDK**, not just photos — it applies equally to card layouts, kanban-style tile boards, etc. There is no CDK-native "grid virtual scroll strategy" as of the current API surface, so this composition is the accepted idiom, and is exactly what several community libraries (e.g., virtual-scroll grid wrappers) do internally.

Interviewer intent: Checks whether the candidate can recognize a genuine gap in CDK's abstraction (item-per-row only) and design the standard row-chunking workaround, rather than assuming CDK grids "just work."

Scenario 18: Server-side rendering (SSR) and virtual scroll — blank page on first paint / SEO concerns

Situation: The marketing team wants a large product catalog (10,000 SKUs) to be crawlable by search engines and to show meaningful content on first paint for Core Web Vitals (SSR via Angular Universal/ `@angular/ssr`). Because `cdk-virtual-scroll-viewport` only renders the items that fit in the (measured) viewport, and the server has no real "viewport height" to measure, SSR output shows an empty or arbitrarily-sized set of items, and crawlers effectively see none of the catalog content.

Question: How do you reconcile virtual scroll with SSR and SEO requirements?

Answer: This is a fundamental tension: virtual scroll's entire value proposition is *not rendering* most content; SEO/SSR wants crawlers and first-paint users to *see* content. The practical resolutions, roughly in order of how teams typically combine them:

1. **Don't virtualize the SSR/SEO-critical view at all — serve a paginated or fully-rendered "crawler-friendly" listing for the initial route, and only switch to virtual scroll for the interactive, already-hydrated client experience.** Concretely: the server-rendered HTML for `/catalog` renders (say) the first 50-100 items as fully real DOM (enough for meaningful first paint and for crawlers to index representative content and links to individual product pages), while pagination links (`<a`

`href="/catalog?page=2">`) provide crawlable access to the rest — crawlers don't need infinite-scroll/virtualized access to all 10,000 in one document; they need *discoverable, crawlable links* to every product, which is better served by either pagination links or a sitemap.

2. **Use a `sitemap.xml` for true SEO discoverability of all 10,000 product pages**, decoupling "can Google find and index every product's own detail page" (solved by the sitemap + each product having its own indexable URL) from "does the catalog *listing* page need to contain all 10,000 in the initial HTML" (it doesn't — Google doesn't require every linked page to appear in one document to index it).
3. **If the catalog listing itself must show meaningfully more than one page's worth for Core Web Vitals/first paint**, render a reasonably-sized initial chunk (e.g., 100-200 items) as plain SSR'd content — not inside `cdk-virtual-scroll-viewport` at all — and only mount the CDK virtual scroll viewport (replacing the static SSR'd block) after hydration, once the browser has a real, measurable viewport:

```
@Component({
  standalone: true,
  imports: [ScrollingModule],
  template: `
    @if (isBrowser() && hydrated()) {
      <cdk-virtual-scroll-viewport itemSize="220" class="viewport">
        <app-product-card *cdkvirtualFor="let p of products(); trackBy: trackById"
[product]="p" />
      </cdk-virtual-scroll-viewport>
    } @else {
      <!-- SSR fallback: plain, fully-rendered, crawlable list of first N items -->
      @for (p of products().slice(0, 100); track p.id) {
        <app-product-card [product]="p" />
      }
    }
  `,
})
export class CatalogComponent {
  private platformId = inject(PLATFORM_ID);
  isBrowser = signal(isPlatformBrowser(this.platformId));
  hydrated = signal(false);
  products = signal<Product[]>([]);

  constructor() {
    afterNextRender(() => this.hydrated.set(true));
  }
}
```

This means SSR output is a real, crawlable, indexable set of `<app-product-card>` elements (good for LCP and for crawlers), and once Angular hydrates in the browser, the component swaps to the interactive virtualized version for smooth infinite scrolling.

4. **Beware hydration mismatches** — the DOM Angular hydration expects to reconcile against must match what the server actually emitted; swapping structure between SSR ("plain list") and client ("virtual scroll viewport") as shown above is a *deliberate* full remount rather than an in-place hydration of the same nodes, which is intentionally simpler and safer than trying to make `cdk-virtual-scroll-viewport` hydration-transparent (Angular's hydration support for CDK virtual scroll specifically has had

evolving/partial support across versions — always verify current version support rather than assuming full non-destructive hydration works transparently).

Interviewer intent: Tests systems-level thinking about a real conflict between two legitimate requirements (performance-via-virtualization vs. SEO/SSR-via-full-render) rather than a narrow CDK API question — strong signal for senior/architect-level roles.

Scenario 19: Auto-scrolling chat/log viewer that should "stick to bottom" for new messages

Situation: A live log-tailing viewer (like `kubectl logs -f`) streams new lines continuously (potentially thousands per minute) into a virtualized list. Desired behavior: if the user is scrolled to the bottom, new lines should auto-scroll to keep the latest line visible (like a real terminal); but if the user has manually scrolled up to read older lines, new incoming lines should *not* yank them back down.

Question: How do you implement "stick to bottom unless the user has scrolled away" behavior correctly with CDK virtual scroll, and what race conditions do you need to guard against?

Answer: The core logic is straightforward — track whether the user is "at/near the bottom" and only auto-scroll if so — but there are real race conditions between "data length changed" and "scroll position measurement" that must be handled carefully at high message throughput:

```
import { Component, signal, effect, viewChild, ElementRef } from '@angular/core';
import { CdkVirtualScrollViewport, ScrollingModule } from '@angular/cdk/scrolling';

@Component({
  standalone: true,
  imports: [ScrollingModule],
  template: `
    <cdk-virtual-scroll-viewport itemSize="20" class="viewport"
    (scrolledIndexChange)="onScroll()">
      <div *cdkVirtualFor="let line of lines(); trackBy: trackByIndex" class="log-line">{{
line }}</div>
    </cdk-virtual-scroll-viewport>
    <div>
      <button class="jump-to-latest" (click)="scrollToBottom(true)">Jump to latest
    </button>
    </div>
  `
})
export class LogViewerComponent {
  viewport = viewChild.required(CdkVirtualScrollViewport);
  lines = signal<string[]>([]);
  isAtBottom = signal(true);
  private readonly BOTTOM_THRESHOLD_PX = 40;

  constructor() {
    // Whenever new lines arrive, only auto-scroll if the user hadn't
    // deliberately scrolled away. Using an effect ties this to the signal,
    // not to a manual "after data update" callback, so it can't be missed.
    effect(() => {
```

```

    const count = this.lines().length; // dependency: re-run whenever new lines arrive
    if (this.isAtBottom()) {
      // Defer to the next microtask so the viewport has processed the new
      // data-length change (via onDataChange) before we ask it to scroll.
      queueMicrotask(() => this.scrollToBottom(false));
    }
  });
}

onScroll() {
  const vp = this.viewport();
  const offset = vp.measureScrollOffset('bottom'); // distance from bottom, if supported
  this.isAtBottom.set(offset <= this.BOTTOM_THRESHOLD_PX);
}

scrollToBottom(smooth: boolean) {
  const total = this.lines().length;
  this.viewport().scrollToIndex(total - 1, smooth ? 'smooth' : 'auto');
  this.isAtBottom.set(true);
}

trackByIndex = (i: number) => i;
}

```

Race conditions and edge cases to call out explicitly:

- **Distinguishing "user-initiated scroll" from "programmatic auto-scroll."**
`scrolledIndexChange` /scroll events fire for *both* — if `onScroll()` naively runs after every scroll event including ones caused by our own `scrollToBottom` call, there's no bug here specifically (both paths correctly conclude "at bottom"), but a common mistake is toggling `isAtBottom` too eagerly on transient intermediate scroll events during the programmatic smooth-scroll animation, which can flicker the "Jump to latest" button. Debouncing `onScroll` slightly (e.g., via a `requestAnimationFrame`-based throttle) smooths this out.
- **High-throughput streams (thousands of lines/minute) flooding change detection.** Pushing every single incoming line as a separate `signal.update()` call causes excessive change-detection/render churn. Batch incoming lines (e.g., buffer them and flush every 100-250ms via `bufferTime` in RxJS, or a manual `requestAnimationFrame`-scheduled flush) before appending to the `lines` signal, so the virtual scroll viewport recalculates its range once per batch, not once per line.
- **Unbounded growth.** A long-running log session appending forever will eventually hold millions of lines in memory even though only ~20 are ever rendered — for genuine log-tailing UX, cap the retained buffer (e.g., keep the most recent 50,000 lines, dropping the oldest) unless there's an explicit requirement to keep the full session history, since virtual scroll bounds *rendering* cost but not *memory* cost of the backing array.
- **"Jump to latest" button UX** is the standard mitigation pattern used by Slack/Discord/terminal-in-browser tools for exactly this scenario — always mention it, since interviewers building chat/log UIs specifically want to hear this pattern by name.

Interviewer intent: Assesses whether the candidate can design a stateful, high-frequency-update UX pattern on top of virtual scroll (not just a static list), including throughput and race-condition considerations — common in observability/chat product interviews.

Scenario 20: Sorting a virtualized table causes a jarring scroll jump

Situation: A virtualized data table (30,000 rows) lets users click a column header to sort. After sorting, the viewport visually jumps to seemingly random content, and the previously-focused/selected row appears to vanish, even though it's still in the data set (just at a new index).

Question: Why does sorting cause a scroll jump in a virtualized list, and how do you make the transition less jarring / preserve context for the user?

Answer: Before virtual scroll, sorting a full-DOM table doesn't change *scroll position* in pixel terms even though row order changes — the user stays looking at the same screen pixels, now showing different (reordered) rows, which is already somewhat disorienting but at least the scrollbar thumb doesn't move. With virtual scroll, the viewport's rendered range is computed from **scroll offset ÷ item size**, i.e., a numeric index into the *current* array — after sorting, the same numeric offset now corresponds to a completely different row (whatever landed at that index post-sort), which is why the visible content appears to "jump" to unrelated data: the scroll position didn't move, but its meaning did.

The two reasonable UX responses, and how to implement each:

Option A — Always reset to top on sort (simplest, often the right call): most data-table UX conventions (spreadsheets, admin tables) already expect sorting to reset your scroll position to the top, since "sorted by X" is a new logical view of the data:

```
onSortChange(column: string) {  
  this.sortColumn.set(column);  
  this.viewport().scrollToIndex(0);  
}
```

Option B — Preserve the user's focused/selected row across the sort (better UX for "I was reviewing row X, now show me where X landed after sorting"):

```
onSortChange(column: string, selectedRowId: string | null) {  
  this.sortColumn.set(column); // triggers re-sort of the underlying `rows` computed signal  
  
  if (selectedRowId) {  
    // After the computed signal re-derives the sorted array, find the row's  
    // NEW index and scroll there instead of leaving the stale numeric offset.  
    queueMicrotask(() => {  
      const newIndex = this.sortedRows().findIndex(r => r.id === selectedRowId);  
      if (newIndex !== -1) {  
        this.viewport().scrollToIndex(newIndex, 'auto');  
      }  
    });  
  }  
}
```

Option B requires the app to already be tracking a "row of interest" (selected/focused row ID) — if there's no such concept (e.g., the user wasn't focused on anything specific, just casually scrolled to some arbitrary depth), there's no principled "new position" to restore to, and resetting to top (Option A) is the honest choice rather than leaving the viewport at a numeric offset that now shows arbitrary, contextually meaningless data.

Also worth mentioning as a related but distinct bug: if `trackBy` is keyed by array **index** rather than row ID, sorting causes Angular to think every row "changed" at each position (since the item at index 5 is now a different logical row), forcing full re-render/re-bind of every visible row's DOM instead of Angular being able to recognize "row ID 482 just moved from index 5 to index 9012 and can be patched, not recreated" — reinforcing, again, that `trackBy` must be identity-based (row ID), never position-based, for virtualized lists undergoing reordering (directly connects to the recycling-correctness theme from Scenario 14).

Interviewer intent: Tests understanding that scroll offset is a *positional*, not *semantic*, pointer into virtualized data, and that meaningful UX requires the app layer to explicitly re-anchor scroll position after any operation that reorders the underlying array.

Scenario 21: Building a custom `VirtualScrollStrategy` for mixed content (headers + items with different fixed sizes)

Situation: A settings/preferences screen has ~2,000 total rows split between section headers (32px each) and setting rows (56px each), in a known, static structure (not dynamically measured). The team wants pixel-perfect, jump-free `scrollToIndex` behavior (needed for an in-page anchor-link sidebar: "jump to Notifications section") without the estimation overhead/inaccuracy of `autosize`.

Question: Walk through implementing a custom `VirtualScrollStrategy` for this case, and explain when writing one is worth it over the built-in strategies.

Answer: Since both row *types* have known, fixed heights (just two different constants), this is a great fit for a custom strategy using precomputed cumulative offsets — deterministic, O(1)-ish lookups, no measurement/estimation error, and reliable arbitrary-index `scrollToIndex` (needed for the sidebar anchor links):

```
import { VirtualScrollStrategy, CdkVirtualScrollViewport } from '@angular/cdk/scrolling';
import { Observable, Subject } from 'rxjs';

export type SettingsRow = { kind: 'header' } | { kind: 'setting' };

export class SettingsVirtualScrollStrategy implements VirtualScrollStrategy {
  private readonly HEADER_HEIGHT = 32;
  private readonly SETTING_HEIGHT = 56;
  private readonly BUFFER_ROWS = 4;

  private viewport: CdkVirtualScrollViewport | null = null;
  private offsets: number[] = []; // offsets[i] = cumulative height through row i
  private readonly _scrolledIndexChange = new Subject<number>();
  scrolledIndexChange: Observable<number> = this._scrolledIndexChange;

  setRows(rows: SettingsRow[]) {
    let sum = 0;
```

```

    this.offsets = rows.map(r => (sum += r.kind === 'header' ? this.HEADER_HEIGHT :
this.SETTING_HEIGHT, sum));
    this.updateContent();
  }

  attach(viewport: CdkVirtualScrollViewport): void {
    this.viewport = viewport;
    this.updateContent();
  }
  detach(): void {
    this._scrolledIndexChange.complete();
    this.viewport = null;
  }
  onContentScrolled(): void { this.updateContent(); }
  onDataLengthChanged(): void { this.updateContent(); }
  onContentRendered(): void {}
  onRenderedOffsetChanged(): void {}

  scrollToIndex(index: number, behavior: ScrollBehavior): void {
    if (!this.viewport) return;
    const offset = index === 0 ? 0 : this.offsets[index - 1];
    this.viewport.scrollToOffset(offset, behavior);
  }

  private updateContent() {
    if (!this.viewport || this.offsets.length === 0) return;
    const scrollOffset = this.viewport.measureScrollOffset();
    const viewportSize = this.viewport.getViewportSize();

    // Binary search for the first row whose cumulative offset exceeds scrollOffset.
    let lo = 0, hi = this.offsets.length - 1, startIndex = 0;
    while (lo <= hi) {
      const mid = (lo + hi) >> 1;
      if (this.offsets[mid] > scrollOffset) { startIndex = mid; hi = mid - 1; }
      else { lo = mid + 1; }
    }

    let endIndex = startIndex;
    while (endIndex < this.offsets.length && this.offsets[endIndex] < scrollOffset +
viewportSize) {
      endIndex++;
    }

    const renderStart = Math.max(0, startIndex - this.BUFFER_ROWS);
    const renderEnd = Math.min(this.offsets.length, endIndex + this.BUFFER_ROWS);

    this.viewport.setTotalContentSize(this.offsets.at(-1) ?? 0);
    this.viewport.setRenderedRange({ start: renderStart, end: renderEnd });
    this.viewport.setRenderedContentOffset(renderStart > 0 ? this.offsets[renderStart - 1] :
0);
    this._scrolledIndexChange.next(startIndex);
  }
}

```



```

// Provide it per-component, not as a global singleton, since it holds
// per-instance state (offsets array) tied to one viewport.
@Component({
  standalone: true,
  imports: [ScrollingModule],
  providers: [{ provide: VIRTUAL_SCROLL_STRATEGY, useClass: SettingsVirtualScrollStrategy }],
  template: `
    <cdk-virtual-scroll-viewport class="viewport">
      @for (row of rows(); track $index) {
        <div [class]="row.kind === 'header' ? 'section-header' : 'setting-row'">...</div>
      }
    </cdk-virtual-scroll-viewport>
  `,
})
export class SettingsPageComponent {
  private strategy = inject(VIRTUAL_SCROLL_STRATEGY) as SettingsVirtualScrollStrategy;
  rows = signal<SettingsRow[]>([]);
  constructor() {
    effect(() => this.strategy.setRows(this.rows()));
  }
}

```

When is writing a custom strategy actually worth it versus using `itemSize / autosize`? Only when **you have accurate, known-ahead-of-time size information the built-ins can't exploit** — a small fixed number of distinct row "types" with known heights (as here), or externally-supplied exact dimensions (e.g., image APIs returning width/height, enabling Pinterest-style masonry-adjacent layouts). If sizes are genuinely unknown until rendered, `autosize` is doing the same job you'd otherwise hand-roll (measure-then-correct), so a custom strategy buys you little. The concrete payoff in this scenario is **reliable, jump-free `scrollToIndex` for the sidebar anchor links** — something `autosize` cannot guarantee for un-rendered, far-off targets, but a precomputed-offset strategy can, since every offset is known in advance rather than estimated.

Interviewer intent: The deepest CDK-internals question in the chapter — filters for candidates who can implement the `VirtualScrollStrategy` interface from scratch and reason about when the investment is justified, a strong senior/staff signal.

Scenario 22: Choosing between `cdk-virtual-scroll-viewport` and a third-party/custom solution for a design-system component library

Situation: A platform team is building a shared, org-wide `<ds-data-table>` component used by dozens of product teams, some needing 50-row tables, others 500,000-row tables, with wildly different row content (simple text cells vs. rich editable cells vs. nested expandable rows). Leadership asks whether to standardize on CDK virtual scroll internally, always render everything and let consuming teams opt into pagination, or build a fully custom windowing engine.

Question: How do you architect a shared component that needs to serve both small and enormous datasets well, and what's your recommendation?

Answer: This is an architecture/API-design question more than a pure virtual-scroll-mechanics question, and a strong answer covers the actual decision framework:

Recommendation: make virtualization an internal, opt-in implementation detail behind a stable public API, not a forced default.

```
@Component({
  selector: 'ds-data-table',
  standalone: true,
  imports: [ScrollingModule],
  template: `
    @if (virtualize()) {
      <cdk-virtual-scroll-viewport [itemSize]="rowHeight()" class="ds-table-viewport">
        <ds-table-row *cdkVirtualFor="let row of data(); trackBy: trackBy()" [row]="row" />
      </cdk-virtual-scroll-viewport>
    } @else {
      @for (row of data(); track trackBy()(0, row)) {
        <ds-table-row [row]="row" />
      }
    }
  `,
})
export class DsDataTableComponent<T> {
  data = input.required<T[]>();
  trackBy = input<(index: number, item: T) => unknown>((i) => i);
  rowHeight = input(48);

  // Auto-decide, but allow explicit override – most consumers should never
  // have to think about this, but a handful with unusual needs (e.g. print
  // views, or SSR/SEO tables per Scenario 18) need the escape hatch.
  private autoVirtualize = computed(() => this.data().length > 200);
  virtualizeOverride = input<boolean | null>(null);
  virtualize = computed(() => this.virtualizeOverride() ?? this.autoVirtualize());
}
```

Reasoning behind each decision:

- **Don't force every consumer into virtual scroll's complexity tax.** Teams with 50-row tables get zero benefit from virtualization and inherit real costs: sticky headers need workarounds (Scenario 3), keyboard nav/a11y needs retrofitting (Scenario 5), editable cells need the recycling-safety discipline (Scenario 14), animations need re-architecting (Scenario 16). A shared component forcing this on everyone is a net loss for the majority of consumers. An automatic threshold (e.g., >200 rows) with an explicit override input gives small-dataset consumers a plain, simple render path "for free," while large-dataset consumers get virtualization without asking for it by name.
- **CDK over a custom windowing engine, by default.** Reinventing `VirtualScrollStrategy`, buffer tuning, resize observation, and scroll-offset math for a design system used by dozens of teams is a large ongoing maintenance burden with real correctness pitfalls (as this entire chapter demonstrates) — CDK is battle-tested, actively maintained by the Angular team, and integrates natively with the rest of `@angular/cdk` (drag-drop, overlay, a11y). Only build custom `VirtualScrollStrategy` implementations for specific, well-understood row-shape needs (Scenario 21) layered *on top of* CDK's viewport, not a full replacement of it.

- **Bake in the hard-won lessons from this chapter as shared, tested infrastructure, not per-team tribal knowledge:** the design system should ship (a) a documented `trackBy` requirement (identity-based, never index-based — Scenarios 11/14/20), (b) a built-in "editable cell" pattern that follows the recycling-safe data flow (Scenario 14), (c) accessibility wiring (`aria-setsize` / `aria-posinset` / `aria-activedescendant`) built into `ds-table-row` so consuming teams get it automatically rather than each team rediscovering Scenario 5's fix independently, and (d) a supported `scrollToRowById` public method (wrapping Scenario 4's pattern) so every consumer gets deep-linking/anchor-jump support without reimplementing it.
- **Document the boundary explicitly:** for genuinely 2D-heavy grids (Scenario 9) or SSR/SEO-critical listings (Scenario 18), the shared component should clearly state it does *not* solve those cases and point to the specific documented alternative pattern, rather than silently producing a broken or suboptimal result for those teams.

Interviewer intent: Tests platform/component-library architecture thinking — whether the candidate can generalize scenario-specific fixes into reusable, documented design-system decisions rather than treating each virtual-scroll problem as a one-off per-feature fix.

Quick Revision Cheat Sheet

- **Use `CdkFixedSizeVirtualScroll (itemSize)` whenever row height is truly constant** — it's O(1) for scroll math and `scrollToIndex`; reach for `autosize` only when heights genuinely can't be known ahead of time, and know it estimates-then-corrects (can jump/flicker).
- **`trackBy` must be identity-based (row ID), never index-based** — this single rule prevents state-leaking-between-rows bugs (Scenario 14), broken diffing after sort/filter (Scenarios 11, 20), and unnecessary full row re-renders.
- **For known, categorizable variable heights** (a few fixed row "types," or externally-supplied exact dimensions), write a custom `VirtualScrollStrategy` with precomputed cumulative offsets instead of relying on `autosize` — it gives reliable, jump-free `scrollToIndex` that estimation-based strategies can't guarantee.
- **CSS `position: sticky` doesn't compose safely with CDK's transform-based content recycling** — implement sticky group headers as a separate, non-virtualized overlay element synced via `scrolledIndexChange`, not as a sticky child inside the recycled content.
- **`scrollToIndex` needs an index, not an ID** — deep-linking to a specific item requires locating (or first fetching/paginating in) the target, then a two-phase "jump then smooth-correct" scroll for estimation-based strategies.
- **Virtual scroll breaks native keyboard/focus/screen-reader behavior** because assistive tech and DOM focus APIs only see the ~20 rendered nodes — fix with `aria-activedescendant` + `aria-setsize` / `aria-posinset`, managing "active index" as app state independent of what's rendered.
- **Server-side pagination + virtual scroll hybrids need the full logical array length up front** (with placeholder/undefined entries) so the scrollbar reflects true total size immediately, plus a sliding cache with eviction so memory stays bounded even at millions of rows.
- **Virtual scroll isn't strictly better than pagination** — pagination wins on deep-linkability, SEO, simplicity, and lower total engineering cost for small/medium datasets; reserve virtual scroll for large, naturally-continuous/scannable content.

- **Nested-scroll-container bugs (modals) are usually CSS/overlay bugs, not virtual-scroll bugs** — `overscroll-behavior: contain` plus building modals on `CdkDialog/Overlay` (which handles background scroll blocking) resolves most bleed-through and stale-position issues.
- **CDK has no native 2D (row × column) virtualization** — for grids, chunk the flat array into row-groups and virtualize the groups (row-level virtualization); for true 2D-heavy cases (spreadsheets), a fully custom renderer is usually required.
- **Eliminate async layout shift at the source** — reserve final item size with known dimensions (`aspect-ratio`, width/height attributes) rather than fighting `autosize`'s measure-then-correct jump after images/content load late.
- **Scroll offset is positional, not semantic** — any operation that reorders the array (sort, filter) invalidates the meaning of the current numeric scroll offset; explicitly decide and implement "reset to top" vs. "re-anchor to the same logical row" rather than leaving it to incidental clamping behavior.
- **Recycled component instances are not equivalent to logical rows** — any row-level state that isn't purely derived from the current `@Input()` /signal value on every change detection pass will leak between rows during recycling; this is the most severe (data-corruption-level) bug class in this chapter.
- **Enter/leave animations must be keyed off genuine data novelty (a persisted "seen" set), not DOM/view insertion** — virtual scroll causes constant view insertion/removal as items scroll in and out, which is indistinguishable from "new" to Angular's animation system unless the app tracks novelty explicitly.
- **Testing virtualized components requires waiting for the async rendered-range update** (`tick()` / `fixture.whenStable()`) in unit tests; scroll-and-poll or driving the app's own "scroll to ID" affordance in E2E) — never assert on total DOM node count as a proxy for total data length.
- **Tune `minBufferPx` / `maxBufferPx` to cover realistic fling-scroll distances**, but treat it as a tradeoff against steady-state memory/render cost, not a free correctness fix — pair with lighter row templates/ `onPush` if blank flashes persist.
- **In a shared/design-system component, make virtualization an internal, opt-in-by-threshold detail behind a stable API** — don't force every consumer to inherit virtual scroll's accessibility, animation, and editable-state complexity when their dataset doesn't need it.

Created By - Durgesh Singh